One Screen, Many Patches: Software Engineering Agents and the Repair of UI-centric Android Bugs

SPARSH PANDEY, University of Central Florida, USA
JINGJING WU, University of Minnesota, USA
JUNAYED MAHMUD, University of Central Florida, USA
NADEESHAN DE SILVA, William & Mary, USA
ANTU SAHA, William & Mary, USA
ATISH KUMAR DIPONGKOR, University of Central Florida, USA
OSCAR CHAPARRO, William & Mary, USA
MATTIA FAZZINI, University of Minnesota, USA
KEVIN MORAN, University of Central Florida, USA

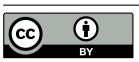
Software engineering (SWE) agents, which pair a large language model with tools for exploring, editing, building, and testing a codebase, have rapidly become the leading approach for automated program repair. However, they are evaluated almost exclusively on library and framework code whose behavior is checked by unit tests. In particular, the research community currently lacks an understanding of how agents perform on bugs in UI-centric programs, such as mobile apps. These bugs carry with them unique challenges, including event-driven control flow, lifecycle-dependent state, and behavior that is only observable on screen.

In this paper, we conduct the first empirical study of SWE agents on UI-centric bugs. We apply three state-of-the-art agents, Claude, Codex, and Gemini, to the 50 real bugs of DroidFixBench under four configurations that vary whether the agent receives fault localization and bug-revealing tests, yielding 600 repair attempts. We analyze these attempts in terms of effectiveness, trajectories and cost, fix complexity, and the root cause of every failed patch. Our results illustrate that agents produce test-passing patches in 74% of attempts, yet only 14% of patches match the developer's fix. The complexity of the developer's fix does not predict whether agents produce a test-passing patch, unlike for LLM-based repair of traditional bugs, and agents instead tend to repair a bug at fewer locations than the developer did. The failures that remain stem mostly from mis-scoped fixes and from Android's activity lifecycle. Additionally, we find that tests, rather than localization, are what change agent behavior, at three to four times the runtime. Our manual analysis of the 162 failures reveals 44 instances that pursue a different repair strategy that could be considered a different possible solution. This illustrates the large universe of potential repair strategies and points to a need for UI-aware tooling and richer oracles as key directions for the agentic repair of UI-centric software.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; *Software maintenance tools*; • **Human-centered computing** → *User interface programming*.

Additional Key Words and Phrases: Automated Program Repair, Software Engineering Agents, Mobile Apps, User Interfaces, Android, Large Language Models, Empirical Study

Authors' Contact Information: Sparsh Pandey, University of Central Florida, Orlando, FL, USA, sparsh.pandey@ucf.edu; Jingjing Wu, University of Minnesota, Minneapolis, MN, USA, wu000295@umn.edu; Junayed Mahmud, University of Central Florida, Orlando, FL, USA, junayed.mahmud@ucf.edu; Nadeeshan De Silva, William & Mary, Williamsburg, VA, USA, kgdesilva@wm.edu; Antu Saha, William & Mary, Williamsburg, Virginia, USA, asaha02@wm.edu; Atish Kumar Dipongkor, University of Central Florida, Orlando, FL, USA, atish.kumardipongkor@ucf.edu; Oscar Chaparro, William & Mary, Williamsburg, VA, USA, oscarch@wm.edu; Mattia Fazzini, University of Minnesota, Minneapolis, MN, USA, mfazzini@umn.edu; Kevin Moran, University of Central Florida, Orlando, FL, USA, kpmoran@ucf.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2027 Copyright held by the owner/author(s).
ACM 2994-970X/2027/7-ARTFSEXXX
https://doi.org/10.1145/XXXXXXX

ACM Reference Format:
Sparsh Pandey, Jingjing Wu, Junayed Mahmud, Nadeeshan De Silva, Antu Saha, Atish Kumar Dipongkor, Oscar Chaparro, Mattia Fazzini, and Kevin Moran. 2027. One Screen, Many Patches: Software Engineering Agents and the Repair of UI-centric Android Bugs. *Proc. ACM Softw. Eng.* 4, FSE, Article FSEXXX (July 2027), 22 pages. https://doi.org/10.1145/XXXXXXX

1 Introduction

Software engineering (SWE) agents have moved, in the space of a few years, from research prototypes to a routine part of how software is built. An agent couples a large language model (LLM) with tools that let it read and search a repository, edit files, and run commands, and it uses these tools in a loop until it judges a task complete [44, 48, 53]. Agents from academic groups and frontier AI labs [2, 11, 32] now triage issues and open pull requests in real projects, and a growing share of the changes merged into open-source repositories originates with them [8, 20, 35]. Nowhere is this shift more visible than in automated program repair (APR). For over a decade, APR techniques generated patches for an already localized bug and relied on an external harness to test them [10, 18, 19, 34]. Now, an agent instead receives an issue and a codebase, localizes the fault itself, and validates its own fix. This had led to staggering progress, as resolution rates on popular benchmarks reaching levels unthinkable just a few years ago [14, 31].

However, the way agents are benchmarked has not kept pace with what they can do. Two problems stand out. The first concerns *what* is measured. The SWE-bench family of benchmarks draws its tasks predominantly from libraries and frameworks whose behavior is checked by unit tests [7, 14, 31, 51]. Absent from this picture is the software that end users actually interact with, namely applications whose behavior is defined by a graphical user interface (UI). Even SWE-bench Multimodal draws its visual tasks from JavaScript UI *frameworks* rather than from the applications built on top of them [49]. The second problem concerns *how* success is measured. A single resolution rate does not show whether the tasks an agent fails share a structure, whether success depends on the information the agent is given, what the agent does with its tools, or why its patches fall short. Recent studies of agent behavior have begun to look past the resolution rate [5, 21, 25], but they do not connect behavior to the kind of software being repaired.

UI-centric software is a particularly revealing domain in which to study agents, because UI code is notoriously hard for *human* developers. A program's UI often accounts for nearly half of the code size, implementation time, and maintenance effort of user-facing software [29], and developers find UI code disproportionately difficult to comprehend, test, and debug [28, 36, 37]. In a UI-centric program, control belongs to the framework. The developer supplies event handlers, and the framework decides when they run in response to user gestures and to lifecycle events, such as a screen being paused or destroyed. The behavior triggered by a single tap can be spread across a layout, a listener, state held in a view model, and a lifecycle callback, and the program's output is a rendered screen rather than a value.

The latest SWE agents are designed to excel at exactly such tasks. An agent can search for the components named in a bug report, read the layout and source files that define them, compile the project, and, when tests are available, run them and react to the result. If tool use of this kind lets agents assemble a real understanding of unfamiliar code, they should cope with UI code where human developers struggle. If it does not, UI code will expose the limits of what tool-mediated comprehension can achieve. Regardless of the potential outcome, there is clear gap in our understanding of how agents handle UI-centric code.

There is a clear need to understand how well SWE agents repair UI-centric bugs, how they go about it, and where they fall short. As such, in this paper we conduct the first empirical study of state-of-the-art SWE agents on UI-centric bugs. We apply Claude (Claude Code with Claude

Opus 4.6), Codex (the Codex CLI with GPT-5.3-Codex), and Gemini (the Gemini CLI with Gemini 3 Flash) [2, 11, 32] to the 50 real bugs of DroidFixBench, which provides real, reproducible UI-centric bugs from open-source Android apps, each with a compilable buggy snapshot, the developer's fix, and GUI tests that expose the bug [24]. We extend the testing harness of this dataset from 2 to 5 tests per bug, complete with a thorough two author manual validation. We run each agent under four configurations that vary whether it receives method-level fault localization and whether it receives the tests, for a total of 600 sandboxed repair attempts. We then measure the *effectiveness* of every attempt, analyze the *trajectories* agents follow and what they *cost*, relate success to the *complexity* of the developer's fix, and determine the *root cause* of every failed patch.

Our findings illustrate both how far agents have come and notable current limitations. Agents produce test-passing patches in 74% of attempts, every bug is plausibly repaired by at least one configuration, and only 9 of the 600 patches fail to build. Yet only 14% of patches match the developer's fix. The gap is filled almost entirely by *alternative fixes*, *i.e.,* patches that pass all five tests, that two authors judged not to overfit them, and that resolve the bug differently from the developer. These account for 356 of the 445 test-passing patches, so measuring agents against the developer's patch, as APR evaluations conventionally do, would credit only about one in five of the fixes they produce. The failures can largely be traced back to issues understanding complex app behavior rather than code syntax. More than 40% of failure root cause labels describe fixes whose scope is wrong. Improper handling of Android's activity lifecycle is the next most common cause and affects all three agents. Finally, tests, rather than localization, change agent behavior, raising the plausible rate by up to 38 percentage points, but at a cost of three to four times the runtime. We also found that the complexity of the developer's fix, which strongly predicts failure for LLM-based repair of traditional bugs [30], does not predict whether agents produce a plausible patch for UI-centric apps. Somewhat surprisingly, agents tend to write patches that are *less* complex than the developer's.

In summary, this paper makes the following main contributions:

- The first comprehensive empirical study of SWE agents on UI-centric bugs, in which three state-of-the-art agents attempt the 50 real bugs of DroidFixBench under four bug information configurations, for a total of 600 repair attempts.
- An analysis of agent trajectories and cost on UI-centric repair that characterizes how agents work, how the information they are given changes their behavior, and how successful and unsuccessful attempts differ.
- An exhaustive root-cause analysis of all 155 non-plausible patches, synthesized into a taxonomy of agent failure modes on UI-centric bugs.
- A replication package containing the infrastructure for running agents on DroidFixBench, the extended test suites, all 600 patches and trajectories, and the labeled data for our analyses [1].

2 Motivating Example

Figure 1 illustrates one of the 50 real bugs in DroidFixBench, taken from Field Book, an open-source app for recording plant phenotyping data in the field.[1] Researchers first import a *field*, which lists the plots of an experiment, and define *traits*, which are the characteristics to record for each plot, such as plant height or the number of flowers. On the Collect screen, they then move from plot to plot and record a value for each trait, for example by tapping the +1 button of a counter to count flowers. According to the bug report, if the user imports a field and opens the Collect screen before defining any traits, pressing the counter's +1 button crashes the app. The crash surfaces in the counter's click handlers, which dereference the trait that the user is currently recording. However, its cause lies on a different screen. Field Book's Config screen opens the Collect screen as

[1] https://github.com/PhenoApps/Field-Book

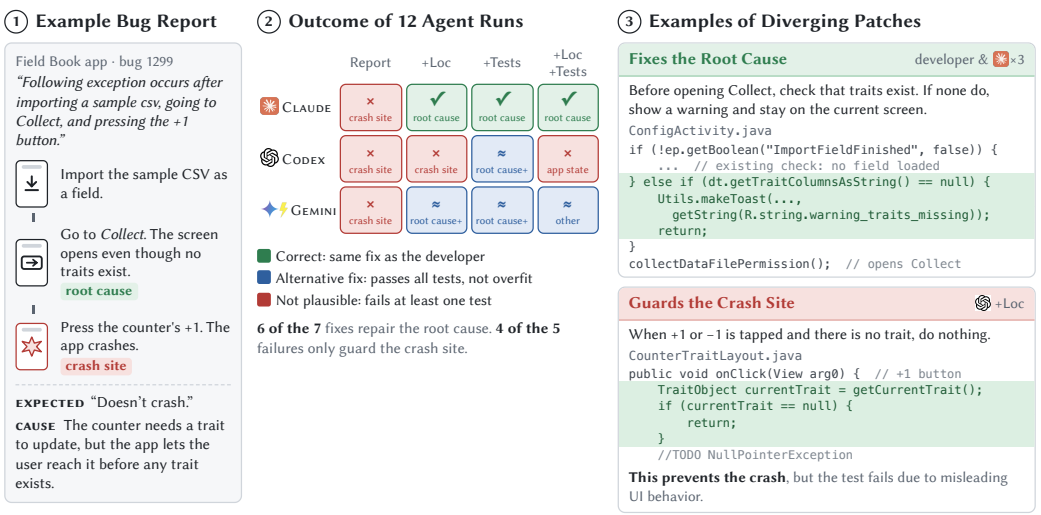


Fig. 1. Example of a UI-centric bug from the Field Book app (bug 1299 in DroidFixBench). If the user opens the Collect screen before creating any traits, pressing the counter's +1 button crashes the app. The developer's patch prevents the user from opening the Collect screen until traits exist. In panel 2, *root cause* marks patches that add the developer's check, *root cause+* marks patches that add it along with extra null checks, *crash site* marks patches that only guard the crash site, *app state* marks a patch that also rewires the app's launch logic, and *other* marks a patch that adds null checks to every trait layout.

soon as a field has been imported, even when no traits have been defined, which leaves the counter without a trait to update. The developer fixed the bug at its root cause, adding a check to the Config screen that keeps the user on that screen and shows a warning when no traits exist. Arriving at this fix requires tracing a crash on one screen back to a navigation decision made on another.

The twelve agent runs on this bug illustrate both what current agents can do and how differently they approach a UI bug. Seven of the twelve runs produced a fix that passes all five of the bug's tests. Claude reproduced the developer's fix in the three configurations that gave it localization, tests, or both. Three further runs, by Codex with tests and by Gemini with either localization or tests alone, added the same kind of check to the Config screen together with additional defensive edits, and thus produced alternative fixes. The remaining alternative fix, by Gemini with both localization and tests, changed the Collect screen itself. Four of the five failed runs, including Claude when it was given only the bug report, took the opposite route and guarded the crash site, adding null checks to the counter's click handlers. These patches remove the crash, but the Collect screen still opens without traits and the counter silently does nothing, so the bug-revealing test, which checks that the user is kept on the Config screen, still fails. We further discuss this relationship between the bug report, the developer's fix, and the test in Section 4.4.

This example illustrates why it is important to look deeper into the performance of agents on UI centric bugs. They now routinely produce patches that pass the tests, and in most runs on this bug they traced the crash to a proper causing screen. However, whether an agent finds that cause depends on the agent and on the information it receives, and four of the seven fixes differ from the developer's, so measuring success against the developer's patch alone would provide an incomplete view of the agent's capabilities. In essence, it is hard to judge or understand an agent's effectiveness on UI-centric Android bugs purely from a "canonical" aggregate pass rate. Instead,

it requires examining how agents arrive at their patches and why those patches fall short, which is what examine in the study presented in this paper.

3 Design of Empirical Study

The goal of this study is twofold. First, we aim to measure how effectively state-of-the-art SWE agents repair real UI-centric Android bugs, and under what conditions. Second, we aim to understand *how* agents arrive at their patches and *why* they fail. With this in mind, we formulate the following research questions:

- **RQ**$_1$: *How effective are SWE agents at repairing real UI-centric Android bugs, and how does the information provided to them affect their effectiveness?*
- **RQ**$_2$: *How do SWE agents behave while attempting to repair UI-centric bugs, what does it cost, and how do the trajectories of successful and unsuccessful attempts differ?*
- **RQ**$_3$: *How does the complexity of the developer's fix relate to repair success?*
- **RQ**$_4$: *Why do agent patches fail, and which failure modes are specific to UI-centric programs?*

For each of the 600 attempts in our study, we collect the final patch and the complete trajectory of actions that produced it. We answer RQ$_1$ from the patches, RQ$_2$ from the trajectories, RQ$_3$ from the patches together with metrics computed over the developer's fix, and RQ$_4$ from a qualitative analysis of every failed patch.

3.1 Benchmark

We use the 50 real bugs of DroidFixBench [24], mined from the issue trackers of 23 open-source Android apps. Each bug includes a compilable snapshot at the buggy commit, the original bug report, the developer's patch, statement-level fault localization, and GUI tests. For this study, we extended the test suite of every bug to five GUI tests: one bug-revealing test that fails on the buggy version and passes on the fixed version, and four regression tests that exercise the surrounding functionality and pass on both. One author wrote the additional tests and a second author validated them by running them on each machine used in our experiments to ensure they agreed on the scenarios and oracles – any disagreements about test behavior were resolved. Of the 50 bugs, 12 are crashes, 28 are output errors, 8 are cosmetic issues, and 2 are navigation issues. According to the fixed methods that DroidFixBench records, the fixes of 27 bugs change a single method and those of 23 change several, adding 414 and removing 290 lines across 63 files in total. We do not use the benchmark's 46 synthetic bugs. They are single-statement mutants, simpler than real bugs and without the cross-cutting fixes that make UI-centric repair difficult, and each would fall into the same complexity class (Section 3.4.3).

3.2 Studied Agents

We study three SWE agents from different frontier labs: Claude, *i.e.,* Claude Code with Claude Opus 4.6 [2, 3]; Codex, *i.e.,* the Codex CLI with GPT-5.3-Codex (`gpt-5.3-codex`) [32, 33]; and Gemini, *i.e.,* the Gemini CLI with the preview release of Gemini 3 Flash (`gemini-3-flash-preview`) [11, 12]. We ran the agents in June 2026, although 10 bugs needed to be rerun after an issue with a test harness due to an environment configuration was discovered. We selected these agents because they represent the state of the art that developers are most likely to use, because they rest on three independently developed model families, and because they expose comparable tools for reading, searching, editing, and running shell commands, which allows us to normalize their trajectories (Section 3.4.2). We used each agent through its official command-line harness with default settings, so that our results reflect the agents as practitioners would encounter them. Because the bugs and their fixes are public, we ran every agent in a sandbox that confines it to the project directory. We did allow internet access, but confirmed no agent cheated in a post analysis on the task via accessing the original issues.

3.3 Experimental Configurations

The information available when a bug is handed to an agent varies in practice, from a bare report to a localized bug with a failing test, although mobile apps very rarely contain test cases [17]. To understand how each kind of information contributes to agent success, and at what cost, we evaluate every agent under a 2×2 factorial design:

- **Localization (Loc):** whether the agent is given method-level fault localization, *i.e.,* the path of each file and the name of each method that the developer's fix modifies, presented as good starting points for revealing the failure.
- **Tests (Tests):** whether the agent is given the bug's five GUI tests and the means to run them on an emulator. The prompt states that one test fails, gives the location of the test sources, and provides commands that run the bug-exposing test and the full suite.

We refer to the four configurations as Report, Report+Loc, Report+Tests, and Report+Loc+Tests. In all of them, the agent receives the repository at the buggy commit and a prompt built from a common template, which we include in our replication package [1]. The prompt states the agent's role, includes the original bug report exactly as the reporter filed it, and instructs the agent to locate the defect, patch it while preserving existing functionality, and confirm that the project builds. We use the original reports, rather than reports restructured by the benchmark's creators, because they reflect what a developer would hand to an agent, even though some omit the expected behavior or the reproduction steps. With tests, the prompt instead asks the agent to make all tests pass. Without tests, we remove the test project and the test-execution script from the agent's working copy. Each agent attempts each bug once per configuration, giving $3 \times 4 \times 50 = 600$ attempts. We chose a single attempt per cell because each attempt is dominated by building and testing on an emulator, and because a single attempt mirrors how a developer uses an agent; we discuss this choice in Section 6.

3.4 Metrics

3.4.1 ***Patch Outcomes.*** A patch is *plausible* if the patched app builds and passes all five tests, and *not plausible* otherwise, in which case we distinguish *build failures* from *test failures* [24, 38]. A plausible patch is *correct* if it is semantically equivalent to the developer's fix, and *overfit* if it passes the tests only by targeting the specific behavior or values that they check. These outcomes were determined manually through independent rating by two authors (Cohen's $\kappa = 0.82$). If a plausible patch is not correct or overfit, they are classified as an *alternative fix*. We therefore treat an alternative fix as a repair of the reported bug that is not equivalent to the developer's. Note that, with tests, plausibility is judged with the same tests that the agent could run, whereas without tests it is judged with tests that the agent never saw.

3.4.2 ***Trajectory Metrics.*** We normalize the three agents' logs into a common sequence of *tool calls* with canonical names (*e.g.,* Claude's `Read`, Codex's `read`, and Gemini's `read_file` all become `read_file`). A deterministic, rule-based procedure then assigns each call one of eight *action labels*. Native file and search tools map to **read**, **search**, or **edit**. Shell commands are classified by a keyword cascade into **test** (the test pipeline or instrumentation tests), **build** (Gradle assemble, build, or compile), **search** (read-only commands such as `grep`, `cat`, or `adb logcat`), or **shell**. Input that Codex feeds to a still-running shell session is labeled **stdin**, and anything else **other**. From the labeled trajectories, we compute per-run metrics of cost (runtime, tool calls, and input, output, and cached tokens, priced at the providers' published rates), of action (the share of each label, files read and edited, builds, and test runs), and of process. For the latter, we map each turn to a **Navigate**, **Reproduce**, **Patch**, or **Validate** phase and derive whether the agent verified the program before its first edit and after its last, how often it verified after editing, and anti-patterns such as edit churn (edits per edited file) and revert pairs. The rules and all 41 metrics are in our replication package.

3.4.3 ***Fix Complexity Metrics.*** We characterize the complexity of each developer fix with two metrics introduced for *multi-hunk* patches, which were strongly predictive of LLM-based repair success on traditional Java bugs [30]. *Hunk divergence* combines the lexical, structural, and file distance between every pair of hunks and scales their mean by $\ln(n)$ for a patch with n hunks, so that it is zero for single-hunk patches and grows as hunks become more numerous, dissimilar, and far apart. The *proximity class* describes where the hunks lie, in five classes of increasing dispersion. A *Nucleus* patch changes a single method, and a *Cluster* patch changes several methods of one file. An *Orbit* patch changes several files in the same package. A patch that spans packages is a *Sprawl* if every pair of its packages shares more than λ leading path segments, and a *Fragment* otherwise, where λ is half the median package depth, rounded down. For example, `tracker/ui` and `tracker/data` share one segment. We compute both metrics with the original authors' tooling, for the developer's fix of every bug and for every agent patch that modifies Java code, with $\lambda = 3$ (the median package depth of our fixes is 7). Both metrics consider only Java hunks, and hunks outside any method in a file, such as imports or field declarations, together count as one additional location. Because a developer's commit can include changes unrelated to the bug, two authors independently reviewed every hunk of the 30 developer commits that change code outside the fix recorded by DroidFixBench, disagreeing on only 2 of the 205 hunks they judged, and we excluded the 50 hunks judged unrelated to the fix, such as license headers, IDE templates, comments, and test changes. Since the pairwise distances depend only on the two hunks they compare, we recompute hunk divergence on the remaining hunks from the distances computed by the tooling. For the proximity class, we resolve the method that encloses each hunk against the source files at the buggy commit, and we correct a defect in the tooling's patch parser that attributed the last hunk of each file to the next file. Across the 50 bugs, hunk divergence has a median of 0.35 (IQR 0.00–0.59), 15 developer fixes have a divergence of zero, 12 of them because they consist of a single hunk, and the five classes contain 16, 17, 4, 13, and 0 bugs, respectively.

3.5 Methodology to Answer Research Questions

3.5.1 ***RQ_1: Effectiveness.*** We report the number and percentage of patches in each outcome category per agent and configuration, as well as the number of bugs repaired by at least one and by all twelve configurations. We determine plausibility automatically by building the patched app and running all five tests. Following the labeling guidelines released with DroidFixBench [24], two authors independently judged each of the 445 plausible patches, agreeing on 419 (Cohen's $\kappa = 0.82$) and resolving the other 26 through discussion. We compare paired per-bug outcomes with exact McNemar tests (two configurations) and Cochran's Q (more than two), apply the Holm correction to pairwise tests, and report 95% Wilson confidence intervals.

3.5.2 ***RQ_2 and RQ_3: Trajectories and Fix Complexity.*** For RQ_2, we compare the trajectory metrics across agents, across configurations, and between plausible and not plausible runs, and we note where a group contains too few runs to support conclusions. We also repeat the outcome split using correctness. For RQ_3, we compute each bug's *repair rate*, *i.e.,* the fraction of the twelve configurations that produced a plausible patch, and relate it to hunk divergence and proximity class with Spearman's ρ over the 50 bugs, overall, per agent, and per configuration, for both plausible and correct repair. We obtain p-values from 20,000 permutations. To compare bugs of different complexity, we also group the developer fixes into easy, medium, and hard tiers with Fisher–Jenks natural breaks, which partitions their hunk divergence into the three contiguous groups with the smallest within-group variance, and compare the tiers with Kruskal–Wallis tests. We also compare the proximity class and hunk divergence of each agent patch with those of the developer's fix.

3.5.3 ***RQ_4: Root-cause Analysis of Failed Patches.*** We analyzed all 155 not plausible patches using open coding [27]. One author labeled every patch with one or more root-cause categories,

examining the diff, the test failure or build error, and, where necessary, the trajectory, and reusing categories from the taxonomy of DroidFixBench [24] where they applied. A second author independently labeled the patches; the two assigned identical category sets to 22 (Cohen's $\kappa = 0.66$), and disagreements generally involved overlapping sets. All disagreements were resolved through discussion. We grouped the 19 resulting categories into six families (Section 4.4).

3.6 Implementation

Before running an agent on a configuration, a script copied the buggy source code of every bug to a fresh working directory and, in configurations without tests, removed the test harness, which consists of an app containing the five tests and a script that builds and installs the app under repair and runs the tests against it. Driver scripts, written in Python for Claude and Codex and in Bash for Gemini, then ran the agent on the 50 bugs of the configuration one after another. They gave each bug the prompt of its configuration and prevented the agent from reading files outside the repository through a read guard. To validate the patches, a script restored the test harness of every bug and ran its five tests, bug by bug, on a Pixel 2 emulator with Android 9 (API 28), the same emulator that the agents could use. We validated every patch with the original five tests in the same emulator environment, regardless of whether the agent had access to the tests, so that changes an agent made to its copy of the tests could not affect the outcome. All agents ran on Apple Silicon Macs with at least 16 GB of memory. We parallelized the experiments by running each agent across three machines, and we validated each patch on the machine that generated it.

Use of AI assistance. An LLM-based coding assistant (Claude Code, separate from our agent runs) helped write our analysis scripts for data extraction, statistics, fix-complexity metrics, and figures, as well as our pipeline for generating and validating patches. No language model labeled any patch; all correctness, overfitting, root-cause, and developer-hunk judgments were made manually by the authors. To guard against errors in the generated code, the scripts verify that they reproduce the values of the original tooling, and a script recomputes the quantitative claims of this paper from the raw data and checks them against the text.

4 Results

We present the results for each research question in the order described in Section 3. Across the 600 repair attempts, 445 (74.2%) produced a plausible patch and 82 (13.7%) produced a patch that matches the developer's fix. Every one of the 50 bugs was plausibly repaired by at least one configuration, 13 bugs were plausibly repaired by all twelve, and 15 bugs were correctly repaired by at least one configuration. Of the 155 attempts that did not produce a plausible patch, 146 failed the tests and only 9 failed to build.

4.1 RQ_1: Effectiveness of Agents on UI-centric Bugs

Figure 2 summarizes the outcome of the attempts per agent and configuration, and Figure 3 shows the outcome of every individual attempt.

Agents produce test-passing patches for most attempts and for every bug. Overall, 445 of the 600 attempts (74.2%; 95% CI 70.5–77.5%) produced a plausible patch, and every one of the 50 bugs was plausibly repaired by at least one of the twelve configurations. The three agents perform similarly in aggregate. Claude produced plausible patches in 77.0% of its attempts, Codex in 76.5%, and Gemini in 69.0%. The most effective single configuration, Codex with tests, produced plausible patches for 47 of the 50 bugs (94%). With both localization and tests, the three agents produced plausible patches for 78% to 90% of the bugs. Only 9 of the 600 agent patches (1.5%) failed to build. Agents are also able to repair bugs whose fixes are large. For example, the developer's fix for bug 1214 of andOTP changes 92 lines, and all twelve configurations produced a plausible patch for it.

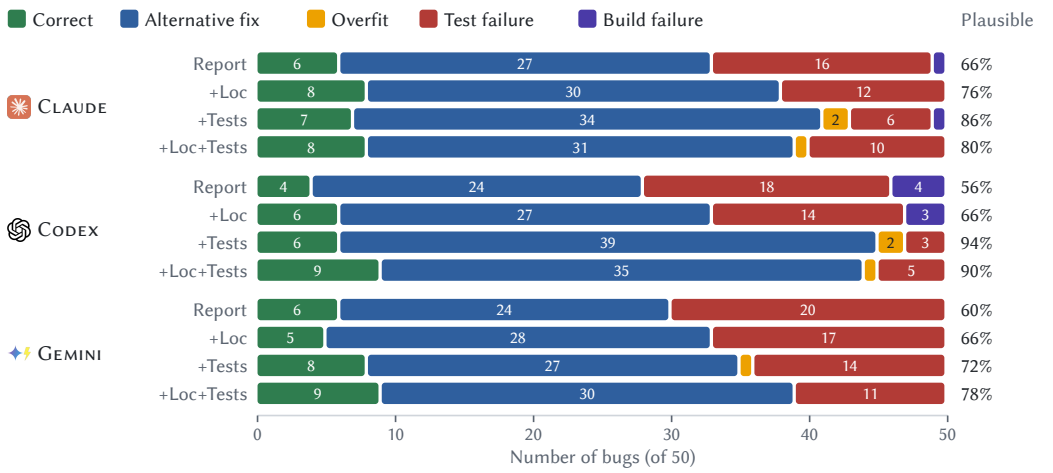


Fig. 2. Outcome of the 50 repair attempts of each agent and configuration. Numbers in the bars count bugs, and the right column gives the plausible rate (correct, alternative, and overfit patches combined).

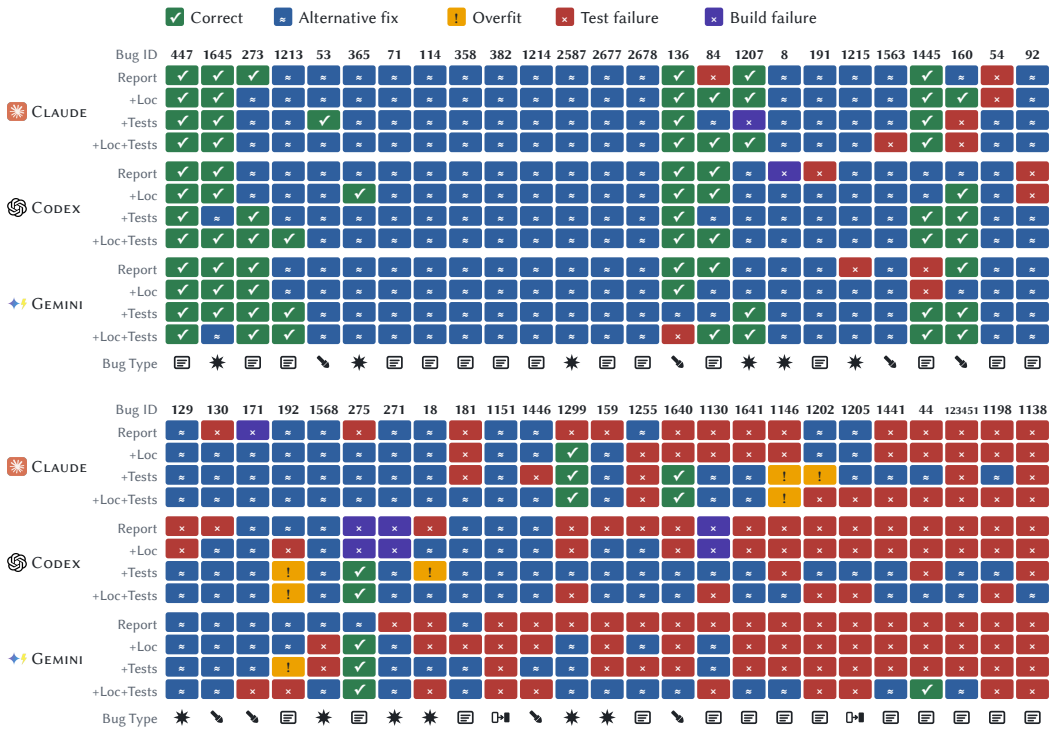


Fig. 3. Outcome of every attempt, per bug (columns) and per agent and configuration (rows). Bugs are sorted by their number of plausible patches and then of correct patches, reading left to right across both halves. The icon below each column gives the bug's type: ✷ crash, output, cosmetic, and navigation.

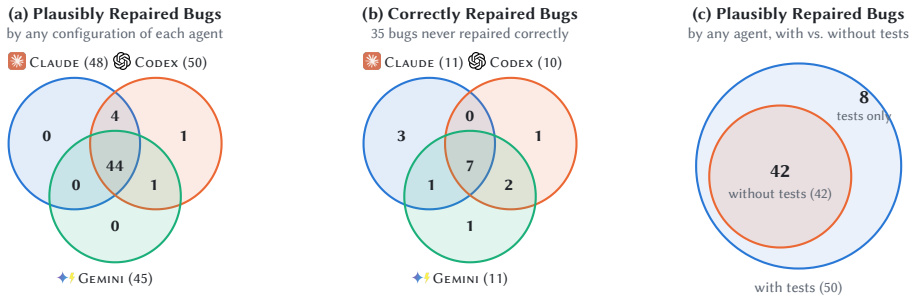


Fig. 4. Overlap of the bugs repaired by the three agents and by the configurations with and without tests.

However, only 14% of patches match the developer's fix, and nearly all others are alternative fixes. Of the 445 plausible patches, 82 (13.7% of all attempts) are correct, 356 are alternative fixes, and only 7 overfit the tests. Only 15 of the 50 bugs were repaired in a manner that matched the developer. Only one bug, bug 447 of URLCheck, was repaired correctly by all twelve configurations. At the other extreme, eight bugs were plausibly repaired by all twelve configurations without a single correct patch.

Bug 114 of Harmonic, a Hacker News client, illustrates how such alternative fixes arise. A long-press on a link should open a sharing menu, but the link's click listener also fires and opens the link. The developer added a flag, set while the menu is showing, that the click listener consults. In ten of the twelve configurations, the agent instead changed the HTML library bundled with the app so that any press longer than the platform's long-press timeout is not treated as a click, a fix that passes all tests but alters how every link in the app behaves. The other two runs, by Codex with localization, ignored clicks for a fixed time window after a long-press.

Overfitting, in contrast, is rare. The 7 overfit patches concern only four bugs, and three of them are Codex and Gemini patches for bug 192 of Markor. The four regression tests per bug likely contribute to this low rate, since a patch that disables or special-cases the surrounding functionality is caught by at least one of them.

Tests, rather than localization, drive plausibility. Because agents in these configurations can run the same tests that later judge their patches, part of this effect reflects the opportunity to check a patch against its eventual oracle; the low overfitting rate indicates that agents rarely exploited it by gaming the tests. Gemini did, however, edit the provided GUI tests in 19 of its 100 runs with tests, mostly to add waits or to make clicks on dialogs optional, and in four runs to remove or change an existing assertion. Claude and Codex never did so. Because every patch was validated against the original tests, these edits could not make a patch plausible, and all four runs that changed an assertion ended in test failures. Providing tests raises the plausible rate by 20 percentage points for Claude (66% to 86%), 38 for Codex (56% to 94%), and 12 for Gemini (60% to 72%). Pooled over agents, tests gave 40 bugs a plausible patch that they lacked without tests and cost only 5 (exact McNemar test, $p < 10^{-7}$), although the effect survives Holm correction only for Codex. Localization alone yields smaller gains of 6 to 10 points, significant only when pooled (17 bugs gained versus 4 lost, $p = 0.007$), and has no measurable effect once tests are available ($p = 0.86$). The agents are statistically indistinguishable in every configuration except Report+Tests, where Codex outperforms Gemini (Cochran's $Q = 10.9$, $p = 0.004$). For correctness, no configuration effect is significant, either for a single agent or pooled. Adding

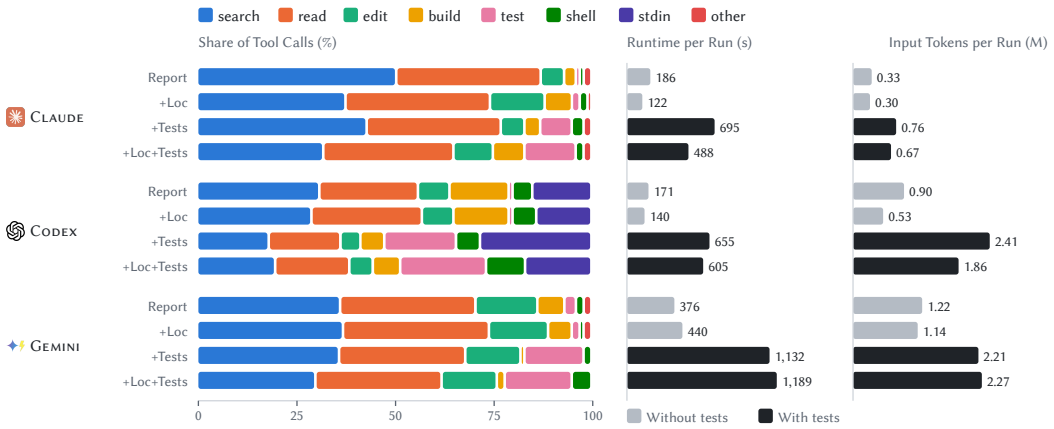


Fig. 5. Agent behavior and cost per configuration, averaged over the 50 runs of each agent and configuration: the share of tool calls carrying each action label (left), and the mean runtime in seconds and input tokens in millions per run (right), with runs with tests in dark bars. All 41 per-run metrics, split by outcome, are in our replication package.

tests on top of localization raises correct patches from 19 to 26 ($p = 0.12$), and the highest correct rate of any configuration, 18%, belongs to Codex and Gemini with both.

The agents fail on largely the same bugs, and some bugs are repaired only with tests. As Figure 4(a) shows, 44 of the 50 bugs were plausibly repaired by all three agents, and no bug was plausibly repaired only by Claude or only by Gemini. The difficulty of a bug is thus largely a property of the bug rather than of the agent, which Figure 3 makes visible as columns that fail across nearly every agent and configuration. Correct repairs are more complementary (Figure 4(b)). Of the 15 bugs ever repaired correctly, 7 were repaired correctly by all three agents and 5 by only one, most often Claude. Finally, every bug that was plausibly repaired without tests was also repaired with them, and 8 bugs were repaired only when the agent could run the tests (Figure 4(c)).

Summary of Findings for RQ$_1$: Agents produce plausible patches in 74.2% of attempts and for every bug at least once, and only 1.5% of patches fail to build. However, only 13.7% of patches match the developer's fix, while 356 of the 445 plausible patches are alternative fixes and only 7 overfit. Providing tests raises plausibility by up to 38 percentage points, whereas localization yields smaller and less consistent gains.

4.2 RQ$_2$: Agent Trajectories

Figure 5 summarizes how the agents behaved across the 600 runs and what their runs cost. We first characterize each agent's working style, then examine how tests and localization change that style, and finally compare successful and unsuccessful attempts.

Each agent has a distinct working style. Claude is search-heavy. Searches make up 32% to 51% of its actions, it edits few files (1.2 to 1.4 per run), and it rarely builds the app on its own (0.9 to 2.2 build attempts per run). Gemini is edit-heavy. It has the highest share of edit actions (14% to 16%), edits the most files (2.4 to 2.7 per run), and revisits them most often, with 1.7 to 2.1 edits per edited file. Codex is build-heavy. Without tests, builds account for 14% to 15% of its actions and 3.1 to 4.0 attempts per run, and it frequently feeds further input to a running shell session, which makes up 14% to 28% of its actions. These styles carry over to cost. Claude consumes the

fewest input tokens by a wide margin, between 0.30M and 0.76M per run, compared with 0.53M to 2.41M for Codex and 1.14M to 2.27M for Gemini.

Providing tests, not localization, is what changes how agents work. With tests, the share of runs that execute any test rises from between 4% and 28% to between 94% and 100%, and the share of actions that run tests rises from 1%–3% to 8%–22%. Agents also verify their edits far more often, with the number of builds and test runs after the first edit growing from 2.6 to 5.0 for Gemini, from 4.3 to 8.3 for Codex, and from 1.1 to 6.5 for Claude. This additional verification is expensive. Relative to the Report configuration, tests increase runtime by a factor of 3.0 for Gemini (376 s to 1,132 s), 3.8 for Codex (171 s to 655 s), and 3.7 for Claude (186 s to 695 s), and they increase input tokens by a factor of 1.8 to 2.7. Without tests, in contrast, agents seldom construct their own verification for the reported behavior. Only 4% to 8% of Codex runs, 18% of Claude runs, and 20% to 28% of Gemini runs executed any test at all. One behavior that tests do not change is Claude's tendency to edit before reproducing the bug. Even with tests available, Claude ran a build or test before its first edit in at most 6% of runs, whereas Gemini did so in every run and Codex in 76% to 98% of runs.

Localization makes some agents leaner but does not change what they do. Its effect is largest for Claude, whose runs shrink from 35 to 15 tool calls, from 7.4 to 2.8 files read, and from 186 s to 122 s. However, Claude with localization also has the highest rate of tool errors (14% of calls). For Codex, localization reduces input tokens from 0.90M to 0.53M per run, while for Gemini it has no measurable effect on run length. In no case does localization alter the composition of actions as tests do.

Unsuccessful attempts are longer and, when tests are available, show signs of thrashing. With tests, failed runs use substantially more tool calls than successful ones: 60 versus 32 for Gemini, 67 versus 45 for Codex, and 81 versus 52 for Claude in the Report+Tests configuration. Failed runs also tend to edit more, revert their own edits more often, and run the tests more often. For example, failed Claude runs in the Report+Tests configuration executed 10.3 test runs on average, compared with 3.5 for successful runs, and failed Gemini runs in the Report+Loc+Tests configuration made 10.0 edits, compared with 4.3. These failures are not caused by skipped validation. Patches whose final edit was never built or tested occur in at most 6% of runs in any configuration. Rather, failed attempts appear to cycle through edits and test runs without converging. As a result, failed runs with tests consume up to 2.6 times as many input tokens as successful ones (3.98M versus 1.52M for Gemini in the Report+Tests configuration). Without tests, the gap between successful and unsuccessful runs is smaller and less consistent, and Claude's failed runs with localization are even shorter than its successful ones (12 versus 16 tool calls), which suggests that it sometimes accepts the localization hint and stops early. We note that some failure groups are small (3 runs for Codex with tests, 5 with localization and tests, and 7 for Claude with tests), and as such its not possible to make strong claims related to them.

Cost is driven by context, not by generation. Output tokens are negligible for every agent, at most 0.02M per run, so the cost of a run is determined almost entirely by the context the agent accumulates. The share of input tokens served from cache varies considerably across agents.

Summary of Findings for RQ_2: The three agents work in distinct styles, from search-heavy (Claude) to edit-heavy (Gemini) and build-heavy (Codex). Providing tests, rather than localization, changes how agents work: nearly every run then executes tests, post-edit verification multiplies, and runtime grows by a factor of three to four. Unsuccessful attempts are longer than successful ones and, with tests, show repeated edit–test cycles that do not converge, rather than skipped validation.



Fig. 6. Repair rates by the proximity class (a) and the complexity tier (b, see Figure 7) of the developer's fix, with statistical tests over the 50 bugs, and the proximity classes of developer fixes and agent patches (c).

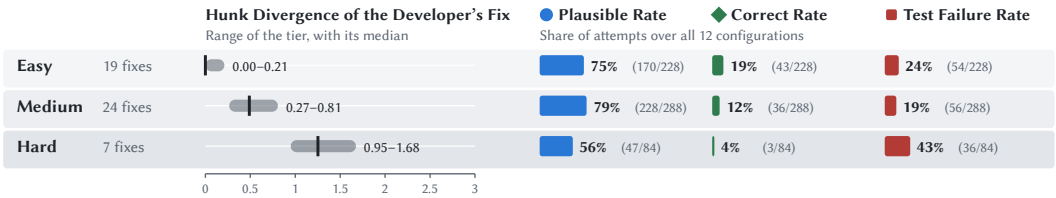


Fig. 7. Complexity tiers of the 50 developer fixes, formed by Fisher–Jenks natural breaks in hunk divergence.

4.3 RQ$_3$: Fix Complexity and Repair Success

Fix complexity does not predict whether agents produce a plausible patch. Figure 6 relates repair success to the complexity of the developer's fix. The plausible repair rate is unrelated to both hunk divergence ($\rho = -0.02$, $p = 0.91$) and proximity class ($\rho = -0.18$, $p = 0.22$), and neither relationship is significant for any agent or configuration. Bugs whose fix spans several packages, which form the most dispersed class among our fixes (Sprawl), are still repaired plausibly in 64% of attempts, compared with 78% for bugs whose fix stays within one method. This contrasts with LLM-based repair of traditional Java bugs, for which both metrics strongly separated fixed from unfixed bugs and no model repaired a bug of the most dispersed class without additional context [30].

Fix complexity has at most a weak relationship with correct repair. The correct repair rate falls from 22% for Nucleus bugs to 7% for Sprawl bugs, but its correlations with both metrics are weak and not significant, even without tests ($\rho = -0.19$, $p = 0.19$ for hunk divergence and $\rho = -0.27$, $p = 0.06$ for proximity class). Figure 7 groups the developer fixes into easy, medium, and hard tiers. Plausible repair is similar for easy and medium bugs (75% and 79%) and lower for the seven hard bugs (56%), and correct repair falls from 19% of attempts on easy bugs to 12% on medium and 4% on hard ones. Neither difference is significant (Kruskal–Wallis $p = 0.23$ and $p = 0.46$). Figure 6(b) splits these rates by whether the agent had the tests. Without tests, agents produced no correct patch in 42 attempts on the hard bugs, compared with 19 of 114 on the easy bugs, and with tests they produced 3. Hard bugs thus remain hard to repair in the developer's way, but we do not have enough data to establish a reliable effect of fix complexity.

Agents produce patches that are more compact than the developer's. For 29 bugs, the median agent patch has a lower hunk divergence than the developer's cleaned fix, and for only 12 a higher one (sign test, $p = 0.012$). Of the agent patches that modify Java code, 43% stay within a single method, compared with 32% of the developer fixes (Figure 6(c)), and for 18 bugs the median agent patch falls in a less dispersed proximity class than the developer's fix, against 7 in a more dispersed one ($p = 0.043$). Agents thus tend to repair a bug at fewer points than the developer did, which likely explains part of the large share of alternative fixes and is consistent with the incomplete fixes among the failures in RQ_4.

Summary of Findings for RQ_3: The complexity of the developer's fix does not predict whether agents produce a plausible patch, and agents plausibly repair the most dispersed fixes as often as the most localized ones. Complexity has at most a weak, non-significant relationship with correct repair. Agents' own patches are more compact than the developer's fixes, suggesting that they often repair a bug at fewer points than the developer did.

4.4 RQ_4: Why Agent Patches Fail

Figure 8 presents the root causes of the 155 agent patches that did not pass the tests. Of these patches, 146 built but failed at least one test, and only 9 failed to build. Our analysis assigned 162 labels across 19 categories, and we group these categories into six families. Below we discuss the main takeaways that emerge from this taxonomy.

Takeaway 1: Agent failures concern program behavior rather than syntax. Only 12 of the 162 labels (7.4%) describe patches that failed to compile or made no change, and Codex accounts for 7 of the 9 compilation errors. This is likely because agents can search the repository for the identifiers they reference and can compile their patches before finishing. Notably, 8 of the 9 compilation errors come from runs without tests, which suggests that building the app as part of running the tests catches the remaining syntactic mistakes.

Takeaway 2: The most common failure is a mis-scoped fix, in which the agent changes the right code but misjudges how much of the behavior to change. Together, incomplete and overreaching fixes account for 69 of the 162 labels (42.6%). Incomplete fixes repair part of the buggy behavior and leave the rest in place. The most frequent case is a patch that addresses only one occurrence of the bug (14 labels), followed by patches that stop a crash without restoring the expected behavior (9 labels), as with the crash-site guards in Figure 1. Overreaching fixes go the other way. They change behavior unrelated to the bug (16 labels) or add a check so strict that it also blocks legitimate behavior (14 labels). These failure modes are often tied to the bug rather than to the agent. For bug 1441 of Anglers' Log, in which the coordinates of a fishing spot are not updated after manual entry, all eight failed patches across the three agents were labeled as having a check that is too strong. Similarly, for bug 123451 of AnkiDroid, in which a warning pop-up is obscured, all nine failed patches

Code	Root Cause	Claude	Codex	Gemini	Total
	Different Repair Strategy 27.2% of labels	**12**	**13**	**19**	**44**
I1	Different solution that the tests reject	12	13	19	44
	Incomplete Fix 22.2% of labels	**11**	**11**	**14**	**36**
I5	Addresses only one occurrence of the bug	9	2	3	14
I7	Fixes crash but not erroneous behavior	1	3	5	9
I8	Does not comply with app-wide context	0	4	3	7
I13	Removes buggy code without a fix	0	0	3	3
I16	Adds checks after buggy behavior	0	1	0	1
I18	Fixed at wrong abstraction level	1	0	0	1
I19	Fixes output but not erroneous behavior	0	1	0	1
	Overreaching Fix 20.4% of labels	**11**	**7**	**15**	**33**
I3	Changes non-buggy behavior	8	2	6	16
I4	Check is too strong	3	4	7	14
I12	Removes app functionality	0	1	2	3
	UI Lifecycle and State† 17.3% of labels	**8**	**10**	**10**	**28**
I2	Improper implementation of activity lifecycle	5	6	9	20
I10	Issue with state initialization/tracking	1	4	1	6
I15	Bug fixed in backend, UI not updated	2	0	0	2
	Build Failure or No Change 7.4% of labels	**3**	**7**	**2**	**12**
I6	Compilation error	2	7	0	9
I11	No diff	1	0	2	3
	Ambiguous Bug Report 5.6% of labels	**1**	**2**	**6**	**9**
I9	Ambiguous expected behavior	1	1	4	6
I14	Ambiguous observed behavior	0	1	1	2
I17	Ambiguous steps to reproduce	0	0	1	1
	All Label Instances	**46**	**50**	**66**	**162**
	Not Plausible Patches	**46**	**47**	**62**	**155**

Claude Codex Gemini
The bar under each category is proportional to its total.

Fig. 8. Root causes of the 155 not plausible patches, by agent, grouped into six families. A patch may carry more than one category (6 do), so the counts sum to 162 labels, and percentages are relative to these 162 labels. The bar under each category is proportional to its total. † means specific to UI-centric programs.

changed behavior unrelated to the bug. This suggests that, for some UI-centric bugs, the extent of the behavior to be changed is precisely what makes the bug hard, regardless of which agent attempts it.

Takeaway 3: More than a quarter of the failures pursue a repair strategy that the tests reject. The single most common category, with 44 labels (27.2%), describes patches that take a different approach from the developer's and fail as a result. Together with the 356 alternative fixes discussed in Section 4.1, they show that agents routinely arrive at strategies other than the developer's. Whether such a strategy counts as a success depends, to a large degree, on what the bug-revealing test checks.

Takeaway 4: Android's activity lifecycle and UI state handling remain a distinct source of failure for all three agents. The lifecycle and state family accounts for 28 labels (17.3%), and improper implementation of the activity lifecycle alone is the second most common category, with 20 labels. Every agent exhibits this failure mode, with 8 labels for Claude and 10 each for Codex and Gemini. Bug 1146 of GPSTest illustrates the difficulty. When the user stops the GNSS receiver, the app should clear the filter text that it displays. The developer's fix ties the visibility of that text to an activity-level flag that records whether the receiver is running. Seven of the eight failed patches for this bug were labeled as lifecycle failures. These patches instead cleared the text in response to the fragment's own state, which the fragment sets again as status updates continue to arrive, so the text is likely to reappear. Fixing such a bug requires reasoning about which component owns a piece of state and when the framework updates it, which is exactly the kind of reasoning that UI programming paradigms make difficult.

Takeaway 5: Some failures originate in the bug report rather than in the agent. Nine labels (5.6%) attribute a failure to an ambiguous description of the expected behavior, observed behavior, or reproduction steps. Bug 1205 of Create PDF is the clearest case: five of its eight failed patches were labeled as stemming from an ambiguous report. Unlike a developer, an agent in our setting cannot ask the reporter for clarification, and Gemini, with six of these nine labels, appears most sensitive to this ambiguity.

Beyond these takeaways, the three agents fail in recognizably different ways. Claude's most frequent failures, after different strategies, are partial fixes that address only one occurrence of a bug (9 labels). Gemini produces the most failures overall (66 labels), leads both the different-strategy (19) and overreaching (15) families, and is the only agent that removed buggy code without replacing it (3 labels). Codex accounts for most of the compilation errors (7 of 9) and most of the state-tracking issues (4 of 6). Providing tests changes the mix of failures as well as their number. Across all agents, runs with tests account for only 53 of the 162 labels, and different-strategy failures fall from 32 labels without tests to 12 with them. Lifecycle failures, in contrast, fall only from 12 to 8, so their share of the remaining failures grows from 11% to 15%. Tests thus appear to help agents correct their choice of strategy far more than they help with lifecycle reasoning. Failure modes also differ by bug type. Crash bugs are the most often repaired plausibly (83% of attempts), and tests help them most (from 74% to 93%), but nearly half of their failure labels (12 of 25) describe incomplete fixes, most often patches that stop the crash without restoring the expected behavior. Output bugs are repaired plausibly in 70% of attempts, and they account for 20 of the 28 lifecycle and state labels. This suggests that the UI-specific failures arise mostly when the agent must keep the information displayed on screen consistent with state that the framework creates, updates, and restores.

Summary of Findings for RQ$_4$: Agent failures on UI-centric bugs are about behavior, not syntax: only 9 of the 155 failed patches do not compile. The most common failure is a mis-scoped fix (42.6% of labels), followed by repair strategies that the tests reject (27.2%). Android's activity lifecycle and UI state account for a further 17.3% of labels, affect all three agents, and are the failure mode that tests reduce the least.

5 Discussion & Future Research Directions

1. UI-centric bugs can have many fixes, and a single developer patch is not a sufficient oracle. Of the 445 plausible patches, 356 are alternative fixes, so matching the developer's patch credits only about one in five fixes, and alternative fixes are not always interchangeable with the developer's, as the app-wide change for bug 114 shows. Benchmarks for UI-centric repair therefore need oracles that sit between test passing and patch matching, such as behavioral equivalence checking at the UI level, differential testing of candidate patches, multiple reference patches, or assisted judging with reported agreement. Evaluations should also report plausible, correct, and alternative rates side by side.

2. Agent failures are about behavior rather than syntax, and most stem from misjudging the extent of a UI behavior. Only 9 of the 600 patches failed to build, while 42.6% of failure labels describe fixes that change too little or too much. Agents' patches are also markedly more compact than the developer's (Section 4.3), suggesting that agents often under-scope their fixes. Agents would likely benefit from support for scoping a fix before editing, for example by enumerating every handler, screen, or occurrence affected by a reported behavior.

3. Android's lifecycle and UI state remain the hardest aspect of UI-centric repair. Lifecycle and state issues account for 17.3% of failure labels, affect all three agents, concentrate on output bugs, and are the failures that tests reduce the least. Directions include exposing lifecycle-aware static analyses to agents as tools, giving agents access to runtime UI state such as view hierarchies and screenshots during validation, and encoding framework-specific guidance in agent harnesses.

4. Tests, not localization, change what agents do, at three to four times the runtime cost. Tests raised the plausible rate by up to 38 percentage points and turned every agent into one that runs tests, whereas, without them, as few as 4% of runs executed any test. When a bug-revealing GUI test can be written or generated, it is therefore the most valuable input a developer can provide, which motivates generating GUI tests and oracles from bug reports as a front end to agentic repair [6, 15, 43, 54].

5. Unsuccessful attempts look different while they are happening. With tests, failed runs used up to twice as many tool calls and up to 2.6 times as many input tokens as successful ones, and they edited and reverted more without converging, rather than skipping validation. Trajectory signals such as edit churn and repeated test failures could therefore drive early stopping, budgeting, or escalation to a developer.

6. The quality of the bug report bounds what an agent can achieve. Nine failure labels trace back to ambiguous reports, and an agent in our setting cannot ask the reporter for clarification. This connects agentic repair to work on interactive bug reporting and report quality [39, 40], and suggests that improving a report before handing it to an agent may be a cheap improvement.

6 Threats to Validity

Internal Validity. The main threats stem from our manual analyses. To mitigate them, two authors independently judged all 445 plausible patches following the labeling guidelines of DroidFixBench ($\kappa = 0.82$, all 26 disagreements resolved), and a second author independently labeled 31 of the 155 failed patches ($\kappa = 0.66$ on exact label sets), with a codebook that reused existing categories where they applied. Finally, an author re-examined the 9 groups of textually identical patches for the same bug that had received different labels across agents or configurations, which changed 7 labels. Identical patches can still differ between overfit and alternative fix, since whether a patch overfits depends on the tests the agent was given. Our action labels come from deterministic keyword rules that may mislabel unusual shell commands; we release the rules and all raw logs. Agents are non-deterministic and we ran each agent once per bug and configuration, so we draw conclusions only at the level of agents and configurations (50 runs each) and test them statistically. Finally, the bugs and fixes are public and may have been seen in training. Our sandbox prevented retrieval during a run but cannot rule out memorization, which would bias the correct rate upward.

External Validity. Our results are a snapshot of three agents at one point in time (Section 3.2), and agent capabilities change quickly. Our 50 bugs come from 23 Java Android apps, and findings may not transfer to Kotlin and Jetpack Compose, iOS, or web applications, although the underlying UI paradigms are shared. We used default harness settings to characterize agents as they are shipped, and tuned prompts might perform better. We do not claim that our results generalize beyond the agents, configurations, and bugs that we studied.

Construct Validity. Plausibility rests on five GUI tests per bug, which is necessary but not sufficient for a correct repair. We complemented the tests with a two-author overfitting review and count non-overfitting patches that differ from the developer's fix as alternative fixes rather than as failures. This review targets patches that game the tests and may miss side effects that the tests do not exercise, such as the app-wide change for bug 114, so some alternative fixes may still be undesirable to maintainers. We did not assess how closely each bug-revealing test encodes the developer's particular fix, so some failures that pursue a different strategy may reflect what the test checks rather than a shortcoming of the agent. With tests, agents could run the same tests that judge plausibility, which the low overfitting rate and the hidden-test configurations help to mitigate. A developer's commit may include changes unrelated to the bug, which would inflate the complexity of the developer's fix. Two authors therefore independently reviewed every hunk of the 30 commits that change code outside the fix recorded by DroidFixBench, disagreeing on only 2 of 205 hunks, and we computed hunk divergence without the hunks judged unrelated. Finally, hunk divergence and proximity class are proxies for fix complexity, and normalizing logs across agents may lose agent-specific nuance, which we handle for Codex's persistent shell with a dedicated action label.

7 Related Work

7.1 Agent-based Program Repair

Over the past two years, program repair has been reshaped by agents that pair an LLM with tools for acting on a repository. Early academic agents established the core design of a language model coupled with an agent–computer interface that can view and search a repository and apply edits [48]. Subsequent designs separate context retrieval from patch generation [53], equip the model with a rich toolset that mirrors the steps a developer would take [4], or decompose repair across multiple cooperating agents [50]. Agents from frontier labs, including Claude, Codex, and Gemini [2, 11, 32], are now in widespread industrial use.

The evaluation of these agents has been driven by SWE-bench, which pairs real GitHub issues from popular repositories with the unit tests that the corresponding pull request made pass [14]. Subsequent benchmarks have curated tasks for reliability [31], broadened the set of languages and repositories [51], lengthened the tasks [7], and introduced issues with visual elements [49]. However, these benchmarks consist of library and framework code, and even the visual tasks of SWE-bench Multimodal concern UI frameworks rather than the applications built with them (Section 1). Our study complements this line of work by evaluating agents on a benchmark whose bugs manifest through the UI of real mobile apps, and whose oracles are GUI tests executed on an emulator.

7.2 Empirical Analyses of Agent Behavior

A smaller body of work has looked beyond agent outcomes to agent behavior. Studies have examined how agents gather and use context, reporting that they tend to retrieve far more than they use and to lose track of information that spans several files [21]. Others have compared agent-authored pull requests with human ones in terms of scope and test coverage [8, 20, 35], and have related features of agent trajectories to success on SWE-bench-style tasks [5, 25]. Our work differs from these studies in two respects. First, we analyze trajectories in a domain where verification requires building an Android app and driving it on an emulator, which changes both the cost and the structure of the agent's feedback loop. Second, we cross the trajectory analysis with the information provided to the agent. This allows us to attribute changes in behavior to the presence of localization or tests rather than to the agent alone.

7.3 Repair and Benchmarks for UI-centric Bugs

Until recently, the automated repair of UI-related defects meant specialized detectors and fixers for particular visual problems. These include techniques for rendering and display issues [22, 23], missing accessibility labels on icons [26], and color-related accessibility issues [52]. Such techniques are built around computer vision and hand-crafted repair templates, and they are therefore limited to the class of defect that each was designed for. The Android bug datasets available to the community were assembled for other purposes, chiefly reproduction, localization, and test generation [16, 41, 45, 47]. As such, they do not include the compilable snapshots, fault locations, tests, and reference fixes that a repair study needs, and the one repair-oriented collection we are aware of is restricted to crashes [42]. DroidFixBench [24] was built to supply all of these ingredients across the full range of UI bug types. The study that introduced it evaluated five traditional APR techniques, including an early open-source agent framework driven by a mid-sized open model [13, 44]. It deferred an evaluation of modern agents, whose trajectories and failure modes deserved dedicated treatment, which this paper provides.

Two concurrent preprints have begun to benchmark LLM-based repair of mobile apps [9, 46]. MobileDev-Bench runs models in a single-pass pipeline rather than as agents and excludes tests that need an emulator, so its unit-level tests cannot observe on-screen behavior. In 85.5% of its tasks,

these tests also reference code added by the developer's fix, so passing largely requires reproducing that fix. AppEval runs tests on a device, but its 200 Android tasks include injected mutations, its reference fixes change a median of two lines, and each task is judged by a single target test. Neither focuses on UI-centric bugs, judges whether test-passing patches are correct, or examines how agents arrive at their patches and why they fail. These gaps matter. Most test-passing patches in our study are alternative fixes that an oracle tied to the developer's code would reject, and many failures involve lifecycle and UI state that unit-level tests do not exercise.

8 Conclusion

In this paper, we presented the first empirical study of state-of-the-art SWE agents on UI-centric bugs, in which three agents attempted the 50 real bugs of DroidFixBench under four information configurations. Agents produce test-passing patches in 74% of attempts, and only 9 of the 600 patches fail to build. However, there are still limitations. Most test-passing patches are alternative fixes that a developer-patch oracle does not credit, and the remaining failures stem from fixes that misjudge the extent of the behavior they change and from Android's lifecycle and UI state. Our findings point to Android-aware tooling for agents, richer oracles for UI behavior, and trajectory-aware control of agents as the next directions for the agentic repair of UI-centric software.

Data Availability

We release our replication package, including the experimental infrastructure for running the agents on DroidFixBench, all 600 patches and trajectories, the normalized action logs, and the labeled data supporting every analysis in this paper [1].

Acknowledgments

[TODO: Acknowledgments and grant numbers.]

References

[1] Anonymous. 2026. Replication Package for "One Screen, Many Patches: Software Engineering Agents and the Repair of UI-centric Android Bugs". https://anonymous.4open.science/r/FSE27-Agentic-UI-Program-Repair.

[2] Anthropic. 2026. Claude Code – Anthropic's Agentic Coding System. https://www.anthropic.com/product/claude-code.

[3] Anthropic. 2026. Claude Opus 4.6 System Card. https://www.anthropic.com/claude-opus-4-6-system-card.

[4] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) *(ICSE '25)*. IEEE Press, 2188–2200. doi:10.1109/ICSE55347.2025.00157

[5] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. https://arxiv.org/abs/2506.18824

[6] Mengzhuo Chen, Zhe Liu, Chunyang Chen, Junjie Wang, Boyu Wu, Jun Hu, and Qing Wang. 2025. Standing on the Shoulders of Giants: Bug-Aware Automated GUI Testing via Retrieval Augmentation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE038 (June 2025), 22 pages. doi:10.1145/3715755

[7] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *arXiv preprint arXiv:2509.16941* (2025).

[8] Atish Kumar Dipongkor, Talank Baral, Wing Lam, and Kevin Moran. 2026. Test Coverage Analysis of Agentic Pull Requests. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME), Visions and Emerging Results Track*. https://arxiv.org/abs/2607.18057

[9] Moshood A. Fakorede, Krishna Upadhyay, A. B. Siddique, and Umar Farooq. 2026. MobileDev-Bench: A Benchmark for Issue Resolution in Mobile Application Development. arXiv preprint arXiv:2603.24946. arXiv:2603.24946 [cs.SE] https://arxiv.org/abs/2603.24946

[10] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey . *IEEE Transactions on Software Engineering* 45, 01 (Jan. 2019), 34–67. doi:10.1109/TSE.2017.2755013

[11] Google. 2026. Gemini CLI – Build, Debug and Deploy with AI. https://geminicli.com.

[12] Google DeepMind. 2025. Gemini 3 Flash Model Card. https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf.

[13] Binyuan Hui et al. 2024. Qwen2.5-Coder Technical Report. *ArXiv* abs/2409.12186 (2024). https://api.semanticscholar.org/CorpusID:272707390

[14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *The Twelfth International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

[15] Jack Johnson, Junayed Mahmud, Oscar Chaparro, Kevin Moran, and Mattia Fazzini. 2025. Generating Failure-Based Oracles to Support Testing of Reported Bugs in Android Apps. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2196–2208. doi:10.1109/ASE63991.2025.00182

[16] Jack Johnson, Junayed Mahmud, Tyler Wendland, Kevin Moran, Julia Rubin, and Mattia Fazzini. 2022. An Empirical Investigation into the Reproduction of Bug Reports for Android Apps. *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER'22)* (2022). doi:10.1109/SANER53432.2022.00048

[17] Pavneet Singh Kochhar, Ferdian Thung, Nachiappan Nagappan, Thomas Zimmermann, and David Lo. 2015. Understanding the Test Automation Culture of App Developers. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. 1–10. doi:10.1109/ICST.2015.7102609

[18] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104

[19] Claire Le Goues, Michael Pradel, Abhik Roychoudhury, and Satish Chandra. 2021. Automatic Program Repair . *IEEE Software* 38, 04 (July 2021), 22–27. doi:10.1109/MS.2021.3072577

[20] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025).

[21] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. *ArXiv* abs/2602.05892 (2026). https://api.semanticscholar.org/CorpusID:285303471

[22] Zhe Liu. 2022. Woodpecker: identifying and fixing Android UI display issues. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 334–336. doi:10.1145/3510454.3522681

[23] Zhe Liu, Chunyang Chen, Junjie Wang, Yuekai Huang, Jun Hu, and Qing Wang. 2020. Owl eyes: spotting UI display issues via visual understanding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual Event, Australia) *(ASE '20)*. Association for Computing Machinery, New York, NY, USA, 398–409. doi:10.1145/3324884.3416547

[24] Junayed Mahmud, Sparsh Pandey, Nadeeshan De Silva, Atish Kumar Dipongkor, Jingjing Wu, Oscar Chaparro, Mattia Fazzini, and Kevin Moran. 2026. Automated Program Repair for UI-centric Android Bugs: How Far Are We? *Proceedings of the ACM on Software Engineering* 3, ISSTA, Article ISSTA181 (2026). doi:10.1145/3832272

[25] Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. 2025. Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories. *arXiv preprint arXiv:2511.00197* (2025).

[26] Forough Mehralian, Navid Salehnamadi, and Sam Malek. 2021. Data-driven accessibility repair revisited: on the effectiveness of generating labels for icons in Android apps. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 107–118. doi:10.1145/3468264.3468604

[27] Matthew B. Miles, A. Michael Huberman, and Johnny Saldaña. 2014. *Qualitative data analysis: a methods sourcebook* (3rd ed.). SAGE Publications, Thousand Oaks, CA.

[28] Brad Myers. 1994. Challenges of HCI Design and Implementation. *Interactions* 1, 1 (Jan. 1994), 73–83. doi:10.1145/174800.174808

[29] Brad A. Myers and Mary Beth Rosson. 1992. Survey on user interface programming. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Monterey, California, USA) *(CHI '92)*. Association for Computing Machinery, New York, NY, USA, 195–202. doi:10.1145/142750.142789

[30] Noor Nashid, Daniel Ding, Keheliya Gallaba, Ahmed E. Hassan, and Ali Mesbah. 2025. Characterizing Multi-Hunk Patches: Divergence, Proximity, and LLM Repair Challenges. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1109/ASE63991.2025.00137

[31] OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/.

[32] OpenAI. 2026. Codex – A Coding Agent that Helps You Build and Ship with AI. https://openai.com/codex/.

[33] OpenAI. 2026. GPT-5.3-Codex System Card. https://openai.com/index/gpt-5-3-codex-system-card/.

[34] Jeff H. Perkins, Sunghun Kim, Sam Larsen, Saman Amarasinghe, Jonathan Bachrach, Michael Carbin, Carlos Pacheco, Frank Sherwood, Stelios Sidiroglou, Greg Sullivan, Weng-Fai Wong, Yoav Zibin, Michael D. Ernst, and Martin Rinard.

2009. Automatically patching errors in deployed software. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) *(SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 87–102. doi:10.1145/1629575.1629585

[35] Dung Pham and Taher A. Ghaleb. 2026. Code Change Characteristics and Description Alignment: A Comparative Study of Agentic versus Human Pull Requests. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR), Mining Challenge Track*. doi:10.1145/3793302.3793565

[36] Mary Beth Rosson, Susanne Maass, and Wendy A. Kellogg. 1986. Designing for designers: an analysis of design practice in the real world. In *Proceedings of the SIGCHI/GI Conference on Human Factors in Computing Systems and Graphics Interface* (Toronto, Ontario, Canada) *(CHI '87)*. Association for Computing Machinery, New York, NY, USA, 137–142. doi:10.1145/29933.30873

[37] David I. Samudio and Thomas D. LaToza. 2022. Barriers in Front-End Web Development. In *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 1–11. doi:10.1109/VL/HCC53370.2022.9833127

[38] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (Bergamo, Italy) *(ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 532–543. doi:10.1145/2786805.2786825

[39] Yang Song, Junayed Mahmud, Nadeeshan De Silva, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2023. BURT: A Chatbot for Interactive Bug Reporting. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 170–174. doi:10.1109/ICSE-Companion58688.2023.00048

[40] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 344–356. doi:10.1145/3540250.3549131

[41] Ting Su, Jue Wang, and Zhendong Su. 2021. Benchmarking automated GUI testing for Android against real-world bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 119–130. doi:10.1145/3468264.3468620

[42] Shin Hwei Tan, Zhen Dong, Xiang Gao, and Abhik Roychoudhury. 2018. Repairing crashes in Android apps. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 187–198. doi:10.1145/3180155.3180243

[43] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William G. J. Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-Driven Automated Whole Bug Report Reproduction for Android Apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1048–1060. doi:10.1145/3650212.3680341

[44] Xingyao Wang et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations (ICLR'25)*. https://openreview.net/forum?id=OJd3ayDDoF

[45] Tyler Wendland, Jingyang Sun, Junayed Mahmud, S M Hasan Mansur, Steven Huang, Kevin Moran, Julia Rubin, and Mattia Fazzini. 2021. Andror2: A Dataset of Manually-Reproduced Bug Reports for Android apps. *Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR'21)* (2021), 600–604. doi:10.1109/MSR52588.2021.00082

[46] Bang Xie, Hao Liu, Zhenyu Shi, Yonghao Zhang, Senjian Zhang, Zhiyuan Peng, Xin Yin, Chenhao Ying, Yuan Luo, Wei Chen, Haiming Jin, Shaocong Long, Xu Liu, and Zhe Peng. 2026. AppEval: A Unified Benchmark for LLM-Based Mobile Application Repair in ArkTS, Swift, and Kotlin. arXiv preprint arXiv:2608.18588. arXiv:2608.18588 [cs.SE] https://arxiv.org/abs/2608.18588

[47] Yiheng Xiong, Mengqian Xu, Ting Su, Jingling Sun, Jue Wang, He Wen, Geguang Pu, Jifeng He, and Zhendong Su. 2023. An Empirical Study of Functional Bugs in Android Apps. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1319–1331. doi:10.1145/3597926.3598138

[48] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 1601, 125 pages. doi:10.52202/079017-1601

[49] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. 2025. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?. In *The Thirteenth International Conference on Learning Representations*.

https://openreview.net/forum?id=riTiq3i21b

[50] He Ye, Aidan Z.H. Yang, Chang Hu, Yanlin Wang, Tao Zhang, and Claire Le Goues. 2025. AdverIntent-Agent: Adversarial Reasoning for Repair Based on Inferred Program Intent. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA062 (June 2025), 23 pages. doi:10.1145/3728939

[51] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. https://arxiv.org/abs/2504.02605

[52] Yuxin Zhang, Sen Chen, Lingling Fan, Chunyang Chen, and Xiaohong Li. 2023. Automated and Context-Aware Repair of Color-Related Accessibility Issues for Android Apps. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1255–1267. doi:10.1145/3611643.3616329

[53] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1592–1604. doi:10.1145/3650212.3680384

[54] Zhaoxu Zhang, Fazle Mohammed Tawsif, Komei Ryu, Tingting Yu, and William G. J. Halfond. 2024. Mobile Bug Report Reproduction via Global Search on the App UI Model. *Proc. ACM Softw. Eng.* 1, FSE, Article 117 (July 2024), 21 pages. doi:10.1145/3660824